

Labo 15 – Verslag JavaScript deel 3

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

docent: **Pim Debaere**

academiejaar **2025–2026**

Inhoud

Opdracht: paragrafen	3
Opdracht: demo slider	4
Opdracht: colorpicker	5
Opdracht: kleurenwisselaar	7
Opdracht: producten	8

Opdracht: paragrafen

Via klasse belangrijk 2^{de} en 4^{de} paragraaf vetgedrukt zetten maar nog geen kleur meegeven.

```
<p>...</p>
<p class="belangrijk">...</p>
<p>...</p>
<p class="belangrijk">...</p>
```

Via javascript willen we elke vetgedrukte paragraaf nu ook een rode tekstkleur geven. Hiervoor maken we een 2^{de} klasse in CSS opvallend om dit toe te kennen.

```
.belangrijk{
    font-weight: bold;
}

.opvallend{
    color: red;
}
```

Om dit nu toe te kennen gebruiken we Javascript

```
const setup = () => {
    let belangrijk = document.getElementsByClassName("belangrijk");

    for (let i = 0; i < belangrijk.length; i++) {
        belangrijk[i].className += " opvallend";
    }
}
window.addEventListener("load", setup);
```

Eerst worden er HTML-elementen opgehaald die in een HTMLCollection komen te zitten.

De elementen die we ophalen zijn de elementen die als class belangrijk hebben.

Via een **for** lus gaan we over al deze elementen lopen en in de string van classes van die elementen opvallend toevoegen. (!!! Hiervoor moet er een spatie tussen staan anders krijg je belangrijkopvallend en dat is geen herkende class in CSS !!!)

De regels in CSS van opvallend gelden nu dus ook voor de elementen die de class belangrijk hadden.

Proin vel quam a purus maximus viverra. Donec mattis enim quam, vitae sodales diam feugiat a. In auctor enim ut placerat viverra. Mauris malesuada pellentesque suscipit. Suspendisse sapien ligula, maximus a tincidunt vitae, venenatis auctor ante. Proin vel bibendum lorem. Ut pellentesque massa metus, eget porta dolor vestibulum et.

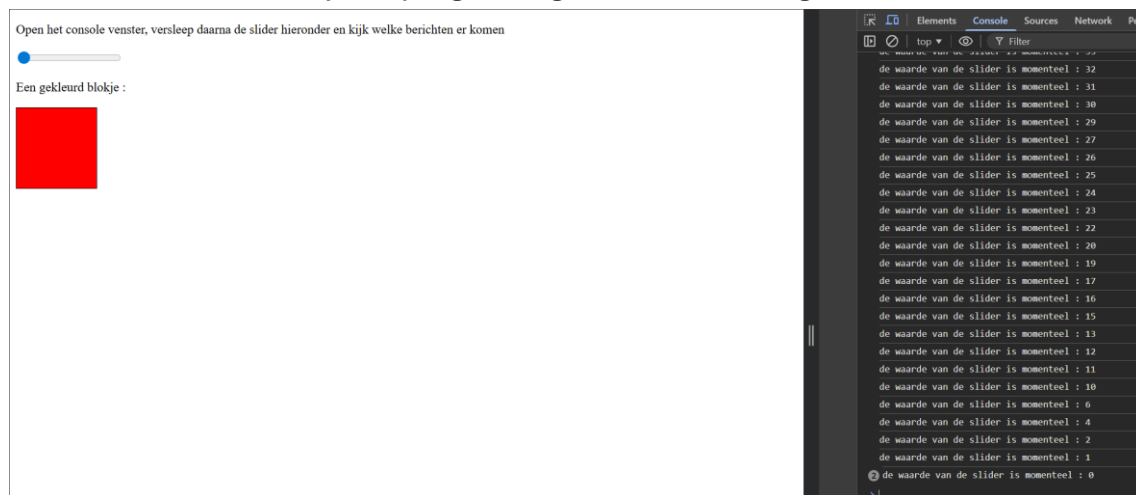
Nulla in massa placerat, accumsan erat et, venenatis est. Cras accumsan justo lorem, sit amet consequat lacus euismod nec. Sed tempor libero nec orci tempor, at porta leo fermentum. Vivamus et ipsum nec massa rutrum tempor id ac metus. Nullam tempus augue nec eros consequat, in hendrerit turpis tempus. Mauris viverra metus a ante pretium, quis finibus sem molestie. Sed vestibulum risus at sem facilis molestie.

Etiam convallis tempor imperdiet. Curabitur nisl turpis, feugiat at efficitur efficitur, bibendum quis quam. Nam sit amet nulla a lorem sollicitudin ullamcorper nec et turpis. Aliquam nec vestibulum mauris, eu viverra mauris. Aliquam ac lectus rhoncus, tincidunt diam at, condimentum ipsum. Integer finibus velit id pharetra aliquam. Sed eleifend aliquet enim, mattis sollicitudin metus ornare at. Nam venenatis sit amet turpis vel maximus. Praesent sodales condimentum mollis.

Nam eleifend tortor libero, eu interdum quam tincidunt ac. Pellentesque rutrum nunc non euismod congue. Donec sed tempus velit. Mauris fringilla eu sem feugiat congue. Suspendisse potenti. Maecenas nec turpis malesuada, porta ante in, consequat leo. Donec condimentum sagittis ante ut eleifend. Nulla sit amet pulvinar mi.

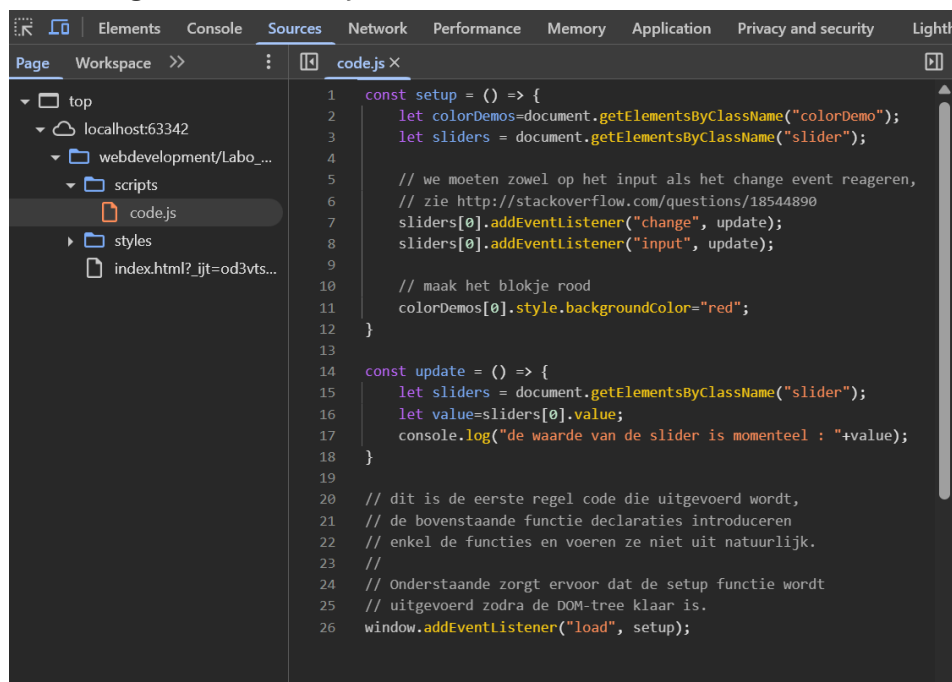
Opdracht: demo slider

Bij het verslepen van de slider kunnen we in de console de continue de waarde van de slider aflezen. Hier is hij nu op 0 geëindigd en deze melding komt 2 keer in de console.



In de setup wordt de event listener 2 keer gekoppeld aan de slider.

Via **change** zal hij 1 keer veranderen en is dit handig om de waarde waarop dat je de slider loslaat zeker te weten. Via **input** zal hij continue veranderen en zelfs als je de slider nog vasthebt zal hij mee veranderen.



In de dezelfde setup zetten we ook het vierkantje al rood via volgende regel.

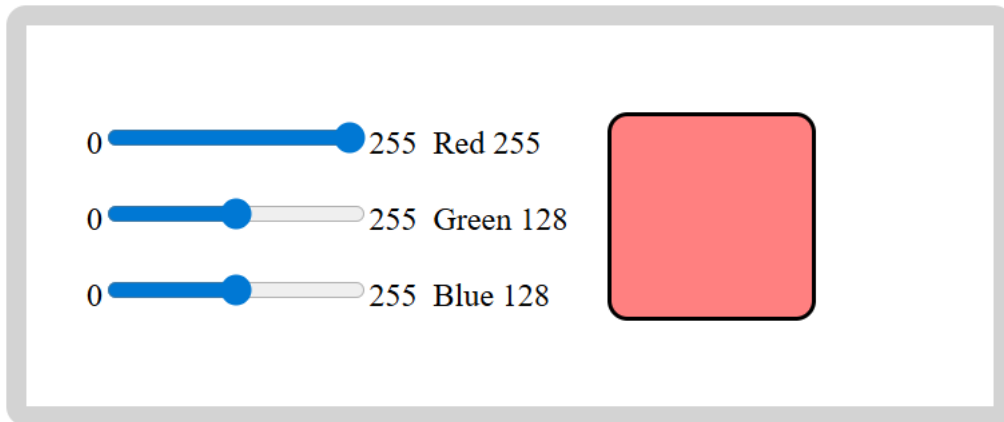
```
colorDemos[0].style.backgroundColor="red";
```

We moeten hier gebruik maken van colorDemos[0] en bij sliders ook sliders[0] omdat het het eerste element is uit een zogezegde HTMLCollection.

Door het gebruik van **getElementsByClassName** krijgen we een HTMLCollection terug. Ook al is dit eigenlijk in sommige gevallen maar 1 element. Om dan het eerste element te kunnen gebruiken moeten we **[0]** gebruiken.

Opdracht: colorpicker

We willen 3 sliders die elk een waarde van de 3 hoofdkleuren voorstelt van RGB. Deze bepalen dan de kleur van het vierkant ernaast.



Om een **slider** aan te maken met min 0 en max 255 gebruiken we volgende regel in HTML

```
<input type="range" class="slider" value="0" min="0" max="255">
```

Achter de benodigde tekst voor de niet-variabele tekst plaats ik nog een **span** op het einde zodat we dan later in Javascript hierin een variabele getal makkelijk kunnen toekennen.

Om te zorgen dat het vierkant op een gelijke hoogte stond heb ik gebruik gemaakt van **flex** en **align-items: center**

Om te starten in de javascript gaan we nu hetzelfde doen zoals vorige oefening maar dan voor alle sliders. Via een for loop overlopen we de HTMLCollection en koppelen we de event listeners aan de sliders.

Op het einde van dit willen we al een update doen van het kleur en voeren we het al op sowieso 1 keer uit.

```
const setup = () => {
  let sliders = document.getElementsByClassName("slider");

  for (let i = 0; i < sliders.length; i++) {
    sliders[i].addEventListener("input", update);
    sliders[i].addEventListener("change", update);
  }

  update();
};
```

Bij het uitvoeren van update halen we nog eens de sliders op omdat we hun waarden nodig hebben, ondertussen halen we ook de spans op die we hadden gemaakt puur om de waarden in te steken. Dit wordt gedaan door de values van de sliders toe te kennen aan de **innerHTML** van de spans.

```
const update = () => {
  let sliders = document.getElementsByClassName("slider");
  let values = document.getElementsByTagName("span");

  let red = sliders[0].value;
  let green = sliders[1].value;
  let blue = sliders[2].value;

  values[0].innerHTML = red;
  values[1].innerHTML = green;
  values[2].innerHTML = blue;

  let colorDemo = document.getElementsByClassName('colorDemo');
  colorDemo[0].style.backgroundColor = `rgb(${red}, ${green}, ${blue})`;
};
```

We willen ook nog de kleur van het vierkant aanpassen dus moeten we het vierkant ophalen die als className colorDemo heeft. Opnieuw is dit een HTMLCollection dus moeten we [0] doen om het element zelf te kunnen verkrijgen.

Via **.style.backgroundColor** kunnen we de achtergrondkleur aanpassen.

Dit gebeurt via een string, in dit geval moeten we wel gebruik maken van **backticks** ``

Via **\${variabeleNaam}** konden we ook de waarden van de sliders in de string krijgen.

Dit kan ook via **concatenatie** met een +.

```
`rgb(${red}, ${green}, ${blue})`  
    of  
`rgb(` + red + `, ` + green + `, ` + blue + `)`;
```

Opdracht: kleurenwisselaar

Er worden 3 knoppen aangemaakt met een bepaalde lay-out.

```
<button>Button 1</button>
<button>Button 2</button>
<button>Button 3</button>
```



In de setup gaan we al deze knoppen ophalen in een HTMLCollection.

Dan gaan we deze overlopen en aan elke knop een event koppelen.

We willen in de methode nog steeds weten welke knop is ingedrukt dus houden we *i* bij als **parameter**. Dus als er wordt gedrukt op een knop zal de functie veranderkleur uitgevoerd worden.

```
const setup = () => {
  let buttons = document.getElementsByTagName("button");

  for (let i = 0; i < buttons.length; i++) {
    buttons[i].addEventListener("click", () => veranderKleur(i));
  }
}
```

We halen opnieuw hier alle knoppen in een HTMLCollection en dan halen we die ene specifieke knop eruit. Dan controleren we welke kleur hij nu heeft en dan passen we die aan naar de andere kleur. (NIET vergeten dat vergelijken met `===` is)

Wit → Blauw

Blauw → Wit

```
const veranderKleur = (index) => {
  let buttons = document.getElementsByTagName("button");
  let button = buttons[index];

  if (button.style.backgroundColor === "lightblue") {
    button.style.backgroundColor = "white";
    button.style.color = "black";
  } else {
    button.style.backgroundColor = "lightblue";
    button.style.color = "white";
  }
}
```



Opdracht: producten

Eerst moeten we een tabel maken in HTML.

Aan de cellen onder de headers van de tabel heb ik een class toegekend om te zorgen dat we ze makkelijk kunnen ophalen in Javascript.

We voegen onderaan ook een button toe.

product	prijs	aantal	btw	subtotaal
product1	10.00 Eur	0	6%	0.00 Eur
product2	15.00 Eur	0	21%	0.00 Eur
product3	12.20 Eur	0	21%	0.00 Eur
totaal				0.00 Eur

Herbereken

De velden in de kolom aantal zijn inputvelden

```
<input type="number" value="0" class="aantal">
```

In de setup halen we de buttons in de HTML op maar we willen er maar 1 dus zetten we bij het toevoegen van een event listener eraan [0]. We controleren op het gedrukt worden. Als dat wordt gedaan dan zal hij de functie berekenen uitvoeren.

```
const setup = () => {  
  let buttons = document.getElementsByTagName("button");  
  buttons[0].addEventListener("click", herbereken);  
};
```

In de functie herbereken gaan we de prijzen, aantallen, btw en subtotaal als HTMLCollections en dan maken we een tussenvariabele om de totale prijs bij te houden.

```
const herbereken = () => {  
  let prijzen = document.getElementsByClassName("prijs");  
  let aantallen = document.getElementsByClassName("aantal");  
  let btw = document.getElementsByClassName("btw");  
  let subtotaal = document.getElementsByClassName("subtotaal");  
  
  let totaal = 0;  
  ...  
};
```


Dan gaan we via een for loop door elke rij lopen en bepalen we de waarde van de velden. Van de niet-inputvelden gebruiken we **textContent** en bij het inputveld gebruiken we **value**. We berekenen dan de subtotalen en dan voegen we de subtotalen toe aan totaal. Dan halen we de cel op voor het totaal en kennen we het resultaat toe aan deze cel. Hetzelfde gebeurde voor de subtotalen die eerder al werden opgehaald. Via **toFixed(2)** kunnen we tot op 2 getallen na de komma afronden.

```
...
for (let i = 0; i < prijzen.length; i++) {
  let prijs = parseFloat(prijzen[i].textContent);
  let aantal = parseFloat(aantallen[i].value);
  let btwPercentage = parseFloat(btw[i].textContent);

  let subtotaal = prijs * aantal * (1 + btwPercentage / 100);

  subtotalen[i].textContent = subtotaal.toFixed(2) + " Eur";
  totaal += subtotaal;
}

document.getElementById("totaal").textContent =
totaal.toFixed(2) + " Eur";
};
```